# Mutex (Mutual Exclusion)

## What is a Mutex?

- Mutex stands for **Mutual Exclusion**.
- It is a **sleeping lock** used to protect critical sections.
- Only **one thread/process** can access the protected resource at a time.
- Preferred over binary semaphores in modern Linux drivers.

## Why Mutex?

Mutex provides:

- Mutual exclusion
- Ownership tracking
- Better performance
- Less overhead
- Better debugging support

## Key Characteristics

### 1. Ownership

The thread that locks the mutex must unlock it.

Thread A -> mutex_lock()

Thread A -> mutex_unlock() ✓

Thread B -> mutex_unlock() ✗

### 2. Binary Lock

Mutex has only two states:

Unlocked (0)
Locked (1)

Cannot be initialized with values greater than 1.

### 3. Sleeping Lock

If the mutex is busy:

mutex_lock()
↓
Task sleeps
↓
Wait Queue
↓
Wakes when mutex becomes free

No CPU cycles are wasted.

## Rules of Mutex

**Rule 1: Process Context Only**

✓ Allowed:

System Calls
Kernel Threads
Read/Write Functions
IOCTL Functions

❌ Not Allowed:

Interrupt Handlers (ISR)
SoftIRQ
Tasklets

Reason:

Mutex may sleep
ISR cannot sleep

## Rule 2: No Recursive Locking

Wrong:

```
mutex_lock(&lock);

mutex_lock(&lock);  // Deadlock
```

A thread cannot lock the same mutex twice.

## Rule 3: Must Unlock Before Exit

Wrong:

```
mutex_lock(&lock);

return;
```

Correct:

```
mutex_lock(&lock);

/* Critical Section */

mutex_unlock(&lock);
```

## Header File

```
#include <linux/mutex.h>
```

MUTEX API

## Dynamic Initialization

```
struct mutex my_mutex;

mutex_init(&my_mutex);
```

## Static Initialization

```
DEFINE_MUTEX(my_mutex);
```

## Lock

```
mutex_lock(&my_mutex);
```

If busy:

Sleep until available

## Interruptible Lock

```
if (mutex_lock_interruptible(&my_mutex))
   return -ERESTARTSYS;
```

Allows interruption by signals.

## Try Lock

```
if (mutex_trylock(&my_mutex))
{
   /* Critical Section */
   mutex_unlock(&my_mutex);
}
```

Does not sleep.

### Unlock

mutex_unlock(&my_mutex);

Must be called by the same thread that acquired the mutex.

### Advantages of Mutex

- Simple to use
- Better performance than binary semaphore
- Ownership tracking
- No busy waiting
- Sleeping allowed
- Preferred in Linux drivers

### Disadvantages of Mutex

- Cannot be used in interrupt context
- Cannot protect very short critical sections efficiently
- Cannot be recursively locked
- Only one owner allowed

### Mutex vs Semaphore

| **Mutex** | **Semaphore** |
|---|---|
| Ownership exists | No ownership |
| Count always 1 | Count can be N |
| Mutual exclusion | Resource counting |
| Preferred in drivers | Used for resource pools |
| Better performance | More generic |

### Mutex vs Spinlock

| Mutex | Spinlock |
|---|---|
| Sleeping lock | Busy-wait lock |
| Long critical sections | Short critical sections |
| Sleeping allowed | Sleeping not allowed |
| Process context only | Process + Interrupt context |
| Preemption enabled | Preemption disabled |

## When to Use Mutex?

Use mutex when:

✓ Need mutual exclusion
✓ Shared buffer protection
✓ File operations
✓ IOCTL handlers
✓ Kernel threads
✓ Sleeping may occur
✓ Long critical sections

## When should a mutex be used?

A mutex should be used when mutual exclusion is required in process context and the critical section may sleep or take a long time to execute. It is the preferred synchronization mechanism for protecting shared resources in modern Linux device drivers.

## Mutex

• Mutual Exclusion
• Sleeping Lock
• Ownership Based
• Process Context Only
• No Recursive Locking
• Preferred over Binary Semaphore

## APIs:

mutex_init()
mutex_lock()
mutex_lock_interruptible()

mutex_trylock()
mutex_unlock()
DEFINE_MUTEX()

## What is Recursive Locking?

Recursive locking means the **same thread tries to acquire the same lock multiple times without releasing it first**.

Linux kernel mutexes are non-recursive. If the same thread attempts to acquire a mutex it already owns, it results in a deadlock because the thread waits indefinitely for itself to release the mutex.

## Which one do you choose between Semaphores and Mutexes?

Start with Mutex and move to Semaphore
only if the strict semantics of Mutexes
are unsuitable.

Start with a Mutex. Use a Semaphore only when the strict ownership rules of a Mutex are unsuitable, such as resource counting or when different threads need to acquire and release the synchronization object.